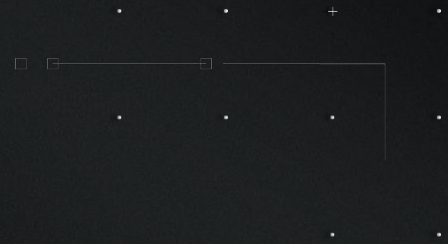


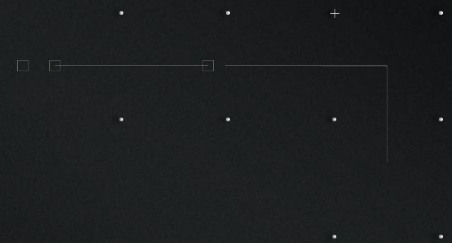
Programação Orientada a Objetos

C#



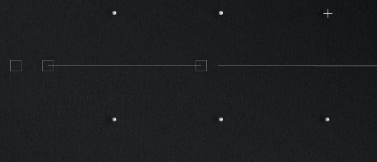
Agenda do dia

1. Sobrecarga
2. Sobrescrita
3. Classe Abstrata



Sobrecarga (overloading)

Ocorre quando você tem duas ou mais coisas (métodos ou funções, por exemplo) com o mesmo nome, mas elas fazem coisas diferentes, geralmente aceitando diferentes tipos ou quantidades de informações. É como ter várias versões de uma mesma coisa, cada uma adaptada para lidar com situações diferentes.



Sobrescrita (overriding)

Acontece quando você tem algo em um lugar (como uma classe), e em outro lugar (normalmente uma classe que herda da primeira), você reescreve ou substitui parte ou todo esse algo para se adequar melhor ao novo contexto. É como personalizar ou adaptar algo já existente para atender às necessidades específicas de um caso particular.



Criação do Projeto

Criação das Classes

Crie as seguintes classes:

Conta

ContaCorrente

ContaPoupanca

Classe Conta.cs

```
9 class Conta
10 {
11     21 references
12     public double Saldo { get; set; }
13
14     2 references
15     public virtual void Depositar(double valor)
16     {
17         Console.WriteLine("Método depositar da classe conta");
18         Saldo += valor;
19         Console.WriteLine($"Depósito de {valor} realizado. Novo saldo: {Saldo}");
20     }
21
22     3 references
23     public virtual void Sacar(double valor)
24     {
25         Console.WriteLine("Método sacar da classe conta");
26         if (valor <= Saldo)
27         {
28             Saldo -= valor;
29             Console.WriteLine($"Saque de {valor} realizado. Novo saldo: {Saldo}");
30         }
31         else
32         {
33             Console.WriteLine("Saldo insuficiente para saque.");
34         }
35     }
36
37     3 references
38     public virtual void CalcularJuros()
39     {
40         Console.WriteLine("Método calcularJuros da classe conta");
41         //Supondo um juros base de 10%
42         Saldo = Saldo - (Saldo * 0.1);
43         Console.WriteLine($"Juros Aplicados. Novo saldo: {Saldo}");
44     }
45 }
```

VIRTUAL indica que um membro é passivo de ser SOBRESCRITO

Classe ContaCorrente.cs

```
9 4 references
10 class ContaCorrente:Conta
11 {
12     3 references
13     public double limiteEspecial { get; set; }
14     1 reference
15     public ContaCorrente()
16     {
17         limiteEspecial = 1000;
18     }
19     0 references
20     public ContaCorrente(double limiteEspecial, double saldo)
21     {
22         this.limiteEspecial = limiteEspecial;
23         this.Saldo = saldo;
24     }
25     3 references
26     public override void Sacar(double valor)
27     {
28         Console.WriteLine("Método sacar da classe conta corrente");
29         if (valor < limiteEspecial + Saldo)
30         {
31             Saldo -= valor;
32             Console.WriteLine($"Saque de {valor} realizado. Novo saldo: {Saldo}");
33         }
34         else
35         {
36             Console.WriteLine("Saldo insuficiente para saque.");
37         }
38     }
39     1 reference
40     public void CalcularJuros(double taxa)
41     {
42         Console.WriteLine("Método calcularJuros da classe conta corrente");
43         Console.WriteLine("Calculando juros da conta corrente.");
44         Saldo = Saldo - (Saldo * (taxa / 100));
45         Console.WriteLine($"Juros Aplicados. Novo saldo: {Saldo}");
46     }
47 }
```

OVERRIDE indica que o método foi SOBRESCRITO

Este método está sendo SOBRECARGADO

Classe ContaPoupanca.cs

```
4 references
9  class ContaPoupanca: Conta
10 {
11     1 reference
12     public ContaPoupanca() { }
13     0 references
14     public ContaPoupanca (double saldo)
15     {
16         Saldo = saldo;
17     }
18     3 references
19     public override void CalcularJuros()
20     {
21         //Supondo um rendimento de 1%
22         Console.WriteLine("Calculando juros da conta poupança.");
23         Saldo = Saldo * 1.01;
24         Console.WriteLine($"Rendimento Aplicado. Novo saldo: {Saldo}");
25     }
26 }
```

Este método está sendo
SOBRESCRITO

```

2  using sobrecargaSobrescrita;
3
4  ContaCorrente contaCorrente = new ContaCorrente();
5  ContaPoupanca contaPoupanca = new ContaPoupanca();
6  double valor = 0;
7
8  while (true)
9  {
10     Console.WriteLine("Escolha uma opção:");
11     Console.WriteLine("1. Depositar na Conta Corrente");
12     Console.WriteLine("2. Sacar da Conta Corrente");
13     Console.WriteLine("3. Depositar na Conta Poupança");
14     Console.WriteLine("4. Sacar da Conta Poupança");
15     Console.WriteLine("5. Calcular Juros da Conta Corrente");
16     Console.WriteLine("6. Calcular Rendimento da Poupança");
17     Console.WriteLine("7. Sair");
18
19     string opcao = Console.ReadLine();
20
21     switch (opcao)
22     {
23     case "1":
24         Console.Write("Digite o valor a depositar na Conta Corrente: ");
25         valor = Convert.ToDouble(Console.ReadLine());
26         contaCorrente.Depositar(valor);
27         break;
28
29     case "2":
30         Console.Write("Digite o valor a sacar da Conta Corrente: ");
31         valor = Convert.ToDouble(Console.ReadLine());
32         contaCorrente.Sacar(valor);
33         break;
34
35     case "3":
36         Console.Write("Digite o valor a depositar na Conta Poupança: ");
37         valor = Convert.ToDouble(Console.ReadLine());
38         contaPoupanca.Depositar(valor);
39         break;
40

```

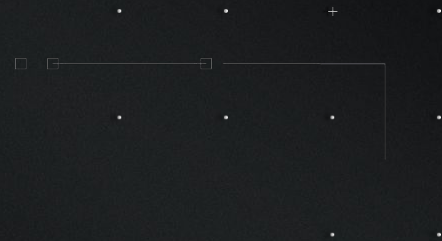
Classe Program.cs

O menuzão!


```
41 case "4":
42     Console.WriteLine("Digite o valor a sacar da Conta Poupança: ");
43     valor = Convert.ToDouble(Console.ReadLine());
44     contaPoupanca.Sacar(valor);
45     break;
46 case "5":
47     valor = 0;
48     Console.WriteLine("Digite a taxa de juros ou 0 para taxa base: ");
49     valor = Convert.ToDouble(Console.ReadLine());
50     if (valor == 0)
51     {
52         contaCorrente.CalcularJuros();
53     }
54     else
55     {
56         contaCorrente.CalcularJuros(valor);
57     }
58     break;
59 case "6":
60     contaPoupanca.CalcularJuros();
61     break;
62 case "7":
63     Console.WriteLine("Saindo do programa.");
64     return;
65
66 default:
67     Console.WriteLine("Opção inválida. Tente novamente.");
68     break;
69 }
70
71 Console.WriteLine(); // Adiciona uma linha em branco para melhor legibilidade
72 }
```


Classe Abstrato

É como um esboço ou um modelo para outras classes. Ela contém características gerais que podem ser compartilhadas por várias classes, mas não pode ser instanciada por si só.



Apenas acrescente a palavra
ABSTRACT a assinatura da classe
Conta para torná-la abstrata.

```
9  2 references  
   abstract class Conta  
10  {  
   21 references  
11  public double Saldo { get; set; }  
12
```

Projeto disponível em:

<https://github.com/profalexresende/sobrecargaSobrescrita.git>

OBRIGADO



profalex.deus@fiap.com.br



/alexsanderresende

FIAP

Copyright © 2018 | Professor (a) Alex Sander Resende de Deus

Todos os direitos reservados. A reprodução ou divulgação total ou parcial deste documento é expressamente proibida sem o consentimento formal, por escrito, do professor/autor.

